

Number: P3372R1
Date: 2024-09-17
Audience: LEWG
Reply-to: [Hana.Dusíková](#)

Table of contents

- [Revision history](#)
- [Introduction and motivation](#)
- [Implementation experience](#)
 - [std::hash](#)
 - [std::less and friends](#)
 - [key & node_handle::key\(\)](#)
 - [non-transient allocations](#)
- [Impact on existing code](#)
- [Intention for wording changes](#)
- [Proposed changes to wording](#)
 - [Feature test macro](#)

constexpr containers and adaptors

This paper proposes marking all containers and adaptors `constexpr`. This will make compile time programming much easier by making normal C++ and `constexpr` C++ more similar.

Revision history

- [R0](#) → R1: Added implementation specific problems with `std::less<T*>`.

Introduction and motivation

All necessary `constexpr` functionality required by the containers to be changed is supported by the language. There is no reason for them not to be `constexpr`, and the standard library should support these in order for them to properly serve as vocabulary types.

The following table shows what this paper proposes, and the status quo:

container / adapter	before	after
<code>std::array</code>	✓	✓
<code>std::deque</code>	✗	✓
<code>std::forward_list</code>	✗	✓
<code>std::list</code>	✗	✓
<code>std::vector</code>	✓	✓
<code>std::map</code>	✗	✓
<code>std::multimap</code>	✗	✓
<code>std::set</code>	✗	✓
<code>std::multiset</code>	✗	✓